

Think Like a Programmer

An Introduction to Creative Problem Solving

by V. Anton Spraul

Errata updated to print 16

Page	Error	Correction	Print corrected
32	As before, we double every second digit, starting with the digit to the right of the check digit, and add the values of all digits, including the check digit, to determine the checksum.	As before, we double every second digit, starting with the digit to the left of the check digit, and add the values of all digits, including the check digit, to determine the checksum.	Print 2
34	<pre>if (doubledDigit > 10) sum = 1 + doubledDigit % 10;</pre>	<pre>if (doubledDigit >= 10) sum = 1 + doubledDigit % 10;</pre>	Print 2
38	<pre>else checksum += 2 * (digit - '0');</pre>	<pre>else checksum += doubledDigitValue(digit - '0');</pre>	Print 2
41	So an input value of 56 in lowercase mode would yield the letter <i>b</i> because 57 modulo 27 is 2 and <i>b</i> is the second letter in the alphabet.	So an input value of 56 in lowercase mode would yield the letter <i>b</i> because 56 modulo 27 is 2 and <i>b</i> is the second letter in the alphabet.	Print 2
46, 47, 48	<pre>Numbered entered</pre>	<pre>Number entered</pre>	Print 3
53	2.1 Using the same rule as the shapes programs from earlier in the chapter (only two output statements—one that outputs the hash mark and one that outputs an end-of-line), write a program that produces the following shape:	2.1 Using only single-character output statements that output a hash mark, a space, or an end-of-line, write a program that produces the following shape:	Print 2
60	If <code>intA > intB</code> , for example, we want to return a positive number, and <code>intA - intB</code> will be positive if <code>intA > intB</code> . Likewise, <code>intA - intB</code> will be negative if <code>intB > intA</code> and will be zero when the two integers are equal.	If <code>intA* > intB*</code> , for example, we want to return a positive number, and <code>intA* - intB*</code> will be positive if <code>intA* > intB*</code> . Likewise, <code>intA* - intB*</code> will be negative if <code>intB* > intA*</code> and will be zero when the two integers are equal.	Print 3
66	Therefore, we initialize it to 0 ❶ and not the value in <code>location[0]</code> .	Therefore, we initialize it to 0 ❶ and not the value in <code>location [0]</code> .	Print 2
71	Here, the variable <code>highPosition</code> ❶ takes the place of <code>highest</code> . Because we aren't directly tracking the grade closest to the average , when it's time to compare the closest grade against the current grade, we use <code>highPosition</code> as a reference into <code>studentArray</code> ❷. If the grade in the current array position is higher, the current position in our processing loop is assigned to <code>highPosition</code> ❸. Once the loop is over, we can access the name of the student with the grade closest to the average using <code>studentArray[highPosition].name</code> , and we can also access any other data related to that student record.	Here, the variable <code>highPosition</code> ❶ takes the place of <code>highest</code> . Because we aren't directly tracking the highest grade , when it's time to compare the highest grade against the current grade, we use <code>highPosition</code> as a reference into <code>studentArray</code> ❷. If the grade in the current array position is higher, the current position in our processing loop is assigned to <code>highPosition</code> ❸. Once the loop is over, we can access the name of the student with the highest grade using <code>studentArray[highPosition].name</code> , and we can also access any other data related to that student record.	Print 3

Page	Error	Correction	Print corrected
72	In a double loop that processes a two-dimensional array, however, the meaningful identifiers help me keep my dimensions and subscripts straight because I can look up and see that I am using <code>agent</code> in the same dimension where I used <code>numAgents</code> in the array declaration.	In a double loop that processes a two-dimensional array, however, the meaningful identifiers help me keep my dimensions and subscripts straight because I can look up and see that I am using <code>agent</code> in the same dimension where I used <code>NUM_AGENTS</code> in the array declaration.	Print 2
72	<pre>double average = (sum + 0.5) / ARRAY_SIZE;</pre>	<pre>double average = sum / ARRAY_SIZE;</pre>	Print 2
73	<pre>int highestAverage = ① arrayAverage(sales[0], 12); for (int agent = 1; agent < NUM_AGENTS; agent++) { int agentAverage = ② arrayAverage(sales[agent], 12);</pre>	<pre>double highestAverage = ① arrayAverage(sales[0], 12); for (int agent = 1; agent < NUM_AGENTS; agent++) { double agentAverage = ② arrayAverage(sales[agent], 12);</pre>	Print 2
77	Furthermore, reading all of the <code>survey responses</code> into the vector required a loop on its own, separate from the loops to process all of the survey responses and find the highest value in the histogram.	Furthermore, reading all of the <code>grades</code> into the vector required a loop on its own, separate from the loops to process all of the survey responses and find the highest value in the histogram.	Print 2
89–90	All the function does is increment count and make a recursive call ①. Recursion is discussed extensively in Chapter 6 but is used here simply to make the chain of function calls as long as possible. The activation record of a function remains on the stack until that function ends. So when the first call is made to <code>stackOverflow</code> from <code>main</code> , an activation record is placed on the runtime stack that cannot be removed until that first function call ends.	All the function does is increment count ② and make a recursive call ③. Recursion is discussed extensively in Chapter 6 but is used here simply to make the chain of function calls as long as possible. The activation record of a function remains on the stack until that function ends. So when the first call is made to <code>stackOverflow</code> from <code>main</code> ④, an activation record is placed on the runtime stack that cannot be removed until that first function call ends.	Print 2
94	Including the <code>allocated</code> memory in our diagrams helps remind us to actually perform the deallocation.	Including the <code>deallocated</code> memory in our diagrams helps remind us to actually perform the deallocation.	Print 2
101	<code>addRecord</code> This function takes a pointer to a collection of student records (<code>a student number and a grade</code>), and it adds a new record with this data to the collection.	<code>addRecord</code> This function takes a pointer to a collection of student records, <code>a student number, and a grade</code> , and it adds a new record with this data to the collection.	Print 2
102	Again, <code>studentCollection</code> is synonymous with <code>node *</code> , but for readability I'm using the <code>studentCollection</code> type only for variables that will refer to the whole list structure.	Again, <code>studentCollection</code> is synonymous with <code>ListNode *</code> , but for readability I'm using the <code>studentCollection</code> type only for variables that will refer to the whole list structure.	Print 2
124	(You might wonder how, if we're validating grades as they are assigned, we could ever have an invalid grade, but <code>remember that our default constructor assigns -1</code> to signal that no legitimate grade has been assigned yet.)	(You might wonder how, if we're validating grades as they are assigned, we could ever have an invalid grade, but <code>we might decide to assign an invalid grade in the constructor</code> to signal that no legitimate grade has been assigned yet.)	Print 2
127	Unlike our node type in Chapter 4, this node doesn't have individual fields for the payload data but rather includes a member of the <code>studentRecord struct</code> type ②.	Unlike our node type in Chapter 4, this node doesn't have individual fields for the payload data but rather includes a member of the <code>studentRecord class</code> ②.	Print 2
128	Now we can turn our attention to the <code>last of the three member functions</code> , <code>recordWithNumber</code> .	Now we can turn our attention to the <code>second member function in the list</code> , <code>recordWithNumber</code> .	Print 2
140	In other words, if we didn't have a class for our linked-list collection of student records, we're still responsible for deleting the nodes in the list when <code>we</code> we're through with them.	In other words, if we didn't have a class for our linked-list collection of student records, we're still responsible for deleting the nodes in the list when we're through with them.	Print 2

Page	Error	Correction	Print corrected
156	<pre>int arraySumRecursiveExtraParams(int integers[], int size, ❶ int sum, ❷ int currentIndex) {</pre>	<pre>int arraySumRecursiveExtraParameters(int integers[], int size, ❶ int sum, ❷ int currentIndex) {</pre>	Print 6
157	<pre>int zeroCountIterative(int numbers[], int size) { int sum = 0; int count = 0;</pre>	<pre>int zeroCountIterative(int numbers[], int size) { int count = 0;</pre>	Print 2
158	<pre>struct listNode {</pre>	<pre>struct listNode {</pre>	Print 6
163	<pre>class binaryTree { public: ❶ int countLeaves(); private: struct binaryTreeNode { int data; binaryTreeNode * left; binaryTreeNode * right; }; typedef treeNode * treePtr; treePtr _root; };</pre>	<pre>class binaryTree { public: ❶ int leafCount(); private: struct binaryTreeNode { int data; binaryTreeNode * left; binaryTreeNode * right; }; typedef binaryTreeNode * treePtr; treePtr _root; };</pre>	Print 2
163	<pre>int numLeaves = bt.countLeaves();</pre>	<pre>int numLeaves = bt.leafCount();</pre>	Print 2
164	<pre>int leafCount(treePtr rootPtr) { if (rootPtr == NULL) return 0; if (rootPtr->right == NULL && rootPtr->left == NULL) return 1; int leftCount = leafCount(rootPtr->left); int rightCount = leafCount(rootPtr->right); return leftCount + rightCount; }</pre>	<pre>int countLeaves(treePtr rootPtr) { if (rootPtr == NULL) return 0; if (rootPtr->right == NULL && rootPtr->left == NULL) return 1; int leftCount = countLeaves(rootPtr->left); int rightCount = countLeaves(rootPtr->right); return leftCount + rightCount; }</pre>	Print 2

Page	Error	Correction	Print corrected
163	<pre>int numLeaves = bt.countLeaves();</pre>	<pre>int numLeaves = bt.leafCount();</pre>	Print 2
165	Because the public function has access to the private data member root, it can pass this along to the recursive function and then return the results back to the client like this: <pre> class binaryTree { public: int publicCountLeaves(); private: struct binaryTreeNode { int data; binaryTreeNode * left; binaryTreeNode * right; }; typedef binaryTreeNode * treePtr; treePtr _root; int privateCountLeaves(treePtr rootPtr); }; ❶ int binaryTree::privateCountLeaves(treePtr rootPtr) { if (rootPtr == NULL) return 0; if (rootPtr->right == NULL && rootPtr->left == NULL) return 1; int leftCount = privateCountLeaves(rootPtr->left); int rightCount = privateCountLeaves(rootPtr->right); return leftCount + rightCount; } ❷ int binaryTree::publicCountLeaves() { ❸ return privateCountLeaves(_root); } </pre> Although C++ would allow both functions to have the same name, for clarity I've used different names to distinguish between the public and private “count leaves” functions. The code in privateCountLeaves ❶ is exactly the same as our previous, independent function countLeaves. The wrapper function publicCountLeaves ❷ is simple. It calls privateCountLeaves, passing the private data member root, and returns the result ❸.	Because the public function has access to the private data member _root, it can pass this along to the recursive function and then return the results back to the client like this: <pre> class binaryTree { public: int publicLeafCount(); private: struct binaryTreeNode { int data; binaryTreeNode * left; binaryTreeNode * right; }; typedef binaryTreeNode * treePtr; treePtr _root; int privateLeafCount(treePtr rootPtr); }; ❶ int binaryTree::privateLeafCount(treePtr rootPtr) { if (rootPtr == NULL) return 0; if (rootPtr->right == NULL && rootPtr->left == NULL) return 1; int leftCount = privateLeafCount(rootPtr->left); int rightCount = privateLeafCount(rootPtr->right); return leftCount + rightCount; } ❷ int binaryTree::publicLeafCount() { ❸ return privateLeafCount(_root); } </pre> Although C++ would allow both functions to have the same name, for clarity I've used different names to distinguish between the public and private “count leaves” functions. The code in privateLeafCount ❶ is exactly the same as our previous, independent function leafCount. The wrapper function publicLeafCount ❷ is simple. It calls privateLeafCount, passing the private data member _root, and returns the result ❸.	Print 2
166	<pre>❶ stack<❷ binaryTreeNode *> nodes;</pre>	<pre>❶ stack<❷ treePtr> nodes;</pre>	Print 2
167	Second, look how many function calls stackBasedCountLeaves makes—for each visit to an interior node (i.e., not a leaf), this function makes four function calls: one each to empty and top, and two to push.	Second, look how many function calls stackBasedCountLeaves makes—for each visit to an interior node (i.e., not a leaf), this function makes up to five function calls: one each to empty, top, and pop, and one or two to push.	Print 2

Page	Error	Correction	Print corrected
178	The first two functions are very simple: higherGrade returns true when the first record has the higher grade, and lowerStudent number returns true when the first record has the lower student number.	The first two functions are very simple: higherGrade returns true when the first record has the higher grade, and lowerStudentNumber returns true when the first record has the lower student number.	Print 2
183	If integerList is a list<int>, populated with integers, and iter is a list<int>::iterator, then we could display all of the integers in the list with the following:	If intList is a list<int>, populated with integers, and iter is a list<int>::iterator, then we could display all of the integers in the list with the following:	Print 2
190	If sra is an array containing arraysize objects of type studentRecord, the resulting code looks like this:	If sra is an array containing arraySize objects of type studentRecord, the resulting code looks like this:	Print 2
192	<pre>1 studentRecord sortArray[arraySize];</pre>	<pre>1 studentRecord * sortArray = new studentRecord[arraySize];</pre>	Print 2
213	<pre>void removeWordsWithoutLetter(list & wordList, char requiredLetter) { list::const_iterator iter;</pre>	<pre>void removeWordsWithoutLetter(list & wordList, char requiredLetter) { list::iterator iter;</pre>	Print 2
214	<pre>void removeWordsWithLetter(list & wordList, char forbiddenLetter) { list::const_iterator iter;</pre>	<pre>void removeWordsWithLetter(list & wordList, char forbiddenLetter) { list::iterator iter;</pre>	Print 2
221	<pre>integerListClass numberList = new integerListClass; numberList.addInteger(15);</pre>	<pre>integerListClass numberList = new integerListClass(); numberList.addInteger(15);</pre>	Print 6